SR UNIVERSITY

AI Assistant Coding
Assignment-12.3

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: Sorting Student Records for Placement Drive

1. Use GitHub Copilot to generate a program that stores student record (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
   a. Quick Sort
   b. Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Prompt:
Generate a Python program to store student records (Name, Roll Number, CGPA). Implement Quick Sort and Merge Sort to sort students by CGPA in descending order. Measure runtime for large datasets and display top 10 students.

Original Code:

Assign 12.3.py > ...

```python
import random
import string
import time
from copy import deepcopy

# ----------------------------
# Data Generation
# ----------------------------
def generate_students(n):
    students = []
    for i in range(n):
        name = ''.join(random.choices(string.ascii_uppercase, k=5))
        roll = f"SRU{i:05d}"
        cgpa = round(random.uniform(5.0, 10.0), 2)
        students.append({"name": name, "roll": roll, "cgpa": cgpa})
    return students

# ----------------------------
# Quick Sort
# ----------------------------
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["cgpa"]
    left = [x for x in arr if x["cgpa"] > pivot]
    middle = [x for x in arr if x["cgpa"] == pivot]
    right = [x for x in arr if x["cgpa"] < pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

Assign 12.3.py > ...

```python
# ----------------------------
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i]["cgpa"] > right[j]["cgpa"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# ----------------------------
# Main Execution
if __name__ == "__main__":
    num_students = 10000
    students = generate_students(num_students)

    # Measure Quick Sort
    start_time = time.time()
    sorted_students_quick = quick_sort(deepcopy(students))
    end_time = time.time()
    print(f"Quick Sort Time: {end_time - start_time:.4f} seconds")

    # Measure Merge Sort
    start_time = time.time()
    sorted_students_merge = merge_sort(deepcopy(students))
    end_time = time.time()
    print(f"Merge Sort Time: {end_time - start_time:.4f} seconds")
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppD
Assign 12.3.py"
Quick Sort Time: 0.0256 seconds
Merge Sort Time: 0.0350 seconds

Top 10 Students by CGPA:
Name: CDJIU, Roll: SRU01712, CGPA: 10.0
Name: QRQXZ, Roll: SRU03473, CGPA: 10.0
Name: NDNGE, Roll: SRU04461, CGPA: 10.0
Name: LHQEN, Roll: SRU04783, CGPA: 10.0
Name: RYJZH, Roll: SRU06210, CGPA: 10.0
Name: EHZPN, Roll: SRU06278, CGPA: 10.0
Name: MFKCM, Roll: SRU09003, CGPA: 10.0
Name: SWSKW, Roll: SRU09662, CGPA: 10.0
Name: HIBHN, Roll: SRU00254, CGPA: 9.99
Name: ZKYAL, Roll: SRU00331, CGPA: 9.99
```

Explanation:

Algorithm	Best Case	Average Case	Worst Case
Quick Sort	**O(n log n)**	**O(n log n)**	**O(n^2)**
Merge Sort	**O(n log n)**	**O(n log n)**	**O(n log n)**

Quick Sort performs faster in practice due to in-place behavior.
Merge Sort guarantees stable performance even for the worst case.

Task 2: Implementing Bubble Sort with AI Comments

Task Description:
Write a Python implementation of Bubble Sort.

Instructions:

- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis

Prompt:
Write a program of Bubble Sort with optimization and analyze time complexity.

Code:

```python
# Task-2: Write a program of Bubble Sort with optimization and analyze time complexity.

def bubble_sort_optimized(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j]["cgpa"] < arr[j + 1]["cgpa"]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

# Time Complexity Analysis:
# The time complexity of the optimized Bubble Sort is O(n) in the best case (when the array is already sorted) and O(
# The average case also has a time complexity of O(n^2). The optimization allows the algorithm to terminate early if

# Example usage:
if __name__ == "__main__":
    students = [
        {"name": "Alice", "roll": "SRU00001", "cgpa": 9.5},
        {"name": "Bob", "roll": "SRU00002", "cgpa": 8.7},
        {"name": "Charlie", "roll": "SRU00003", "cgpa": 9.0},
        {"name": "David", "roll": "SRU00004", "cgpa": 7.5},
        {"name": "Eve", "roll": "SRU00005", "cgpa": 9.2}
    ]

    sorted_students = bubble_sort_optimized(students)
    print("Students sorted by CGPA in descending order:")
    for student in sorted_students:
        print(f"Name: {student['name']}, Roll: {student['roll']}, CGPA: {student['cgpa']}")
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppDa
Assign 12.3.py"
Students sorted by CGPA in descending order:
Name: Alice, Roll: SRU00001, CGPA: 9.5
Name: Eve, Roll: SRU00005, CGPA: 9.2
Name: Charlie, Roll: SRU00003, CGPA: 9.0
Name: Bob, Roll: SRU00002, CGPA: 8.7
Name: David, Roll: SRU00004, CGPA: 7.5
```

Explanation:

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly swaps adjacent elements if they are in the wrong order. It continues making passes through the list until no swaps are needed, meaning the list is sorted. An optimization can stop the algorithm early if the list is already sorted. It is easy to understand and implement but inefficient for large datasets. Therefore, it is mainly useful for learning purposes or very small inputs.

Task 3: Quick Sort and Merge Sort Comparison

Task Description:
Implement Quick Sort and Merge Sort using recursion.

- **Provide AI with partially completed functions for recursion.**
- **Ask AI to complete the missing logic and add docstrings.**
- **Compare both algorithms on random, sorted, and reverse-sorted lists.**

Prompt:
Implement a linked list class with the following methods: insert, delete, and search, Add inline comments explaining pointer manipulation, Highlight tricky

parts of insertion and deletion. Also Suggest edge case test scenarios (empty list, single node,deletion at head)

Code:

Assign 12.3.py > ...

```python
# Code for Quick Sort and Merge Sort is already provided.

#----------------------------
# Quick Sort
# ----------------------------
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["cgpa"]
    left = [x for x in arr if x["cgpa"] > pivot]
    middle = [x for x in arr if x["cgpa"] == pivot]
    right = [x for x in arr if x["cgpa"] < pivot]
    return quick_sort(left) + middle + quick_sort(right)

# ----------------------------
# Merge Sort
# ----------------------------
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i]["cgpa"] > right[j]["cgpa"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppDa
Assign 12.3.py"
Students sorted by CGPA in descending order:
Name: Alice, Roll: SRU00001, CGPA: 9.5
Name: Eve, Roll: SRU00005, CGPA: 9.2
Name: Charlie, Roll: SRU00003, CGPA: 9.0
Name: Bob, Roll: SRU00002, CGPA: 8.7
Name: David, Roll: SRU00004, CGPA: 7.5
```

Explanation:

Input Type	Quick Sort	Merge Sort
Random	**O(n log n)**	**O(n log n)**
Sorted	**$O(n^2)$**	**O(n log n)**
Reverse	**$O(n^2)$**	**O(n log n)**

Quick Sort:

- Best Case: O(n log n) when the pivot divides the array into two equal halves.

- Worst Case: O(n^2) when the smallest or largest element is always chosen as the pivot (e.g., when the array is already sorted).

- Average Case: O(n log n) for random pivots.

Merge Sort:

- Best Case: O(n log n) regardless of the input, as it always divides the array into two halves.

- Worst Case: O(n log n) for all cases, as it always divides the array into two halves and processes each half independently.

- Average Case: O(n log n) for all cases, as it always divides the array into two halves and processes each half independently.

Task 4: Real-Time Application – Inventory Management System

Task Description:

- **Use AI to suggest the most efficient search and sort algorithms for this use case.**
- **Implement the recommended algorithms in Python.**
- **Justify the choice based on dataset size, update frequency, and performance requirements.**

Prompt:

Suggest the most efficient search and sort algorithms for this use case and Implement the recommended algorithms in Python.

Code:

```
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity
    def __repr__(self):
        return f"Product(ID: {self.product_id}, Name: {self.name}, Price: {self.price}, Sto
class Inventory:
    def __init__(self):
        self.products_by_id = {}
        self.products_by_name = []
    def add_product(self, product):
        self.products_by_id[product.product_id] = product
        self.products_by_name.append(product)
    def search_by_id(self, product_id):
        return self.products_by_id.get(product_id, None)
    def search_by_name(self, name):
        return [product for product in self.products_by_name if product.name == name]
    def sort_by_price(self):
        return sorted(self.products_by_name, key=lambda x: x.price)
    def sort_by_stock_quantity(self):
        return sorted(self.products_by_name, key=lambda x: x.stock_quantity)
# Example usage
if __name__ == "__main__":
    inventory = Inventory()
    inventory.add_product(Product("P001", "Laptop", 999.99, 10))
    inventory.add_product(Product("P002", "Smartphone", 499.99, 50))
    inventory.add_product(Product("P003", "Headphones", 199.99, 30))

    print("Search by ID (P002):", inventory.search_by_id("P002"))
    print("Search by Name (Laptop):", inventory.search_by_name("Laptop"))

    print("\nProducts sorted by Price:")
    for product in inventory.sort_by_price():
        print(product)

    print("\nProducts sorted by Stock Quantity:")
    for product in inventory.sort_by_stock_quantity():
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppData\Local\Programs\Pyt
Assign 12.3.py"
Search by ID (P002): Product(ID: P002, Name: Smartphone, Price: 499.99, Stock: 50)
Search by Name (Laptop): [Product(ID: P001, Name: Laptop, Price: 999.99, Stock: 10)]

Products sorted by Price:
Product(ID: P003, Name: Headphones, Price: 199.99, Stock: 30)
Product(ID: P002, Name: Smartphone, Price: 499.99, Stock: 50)
Product(ID: P001, Name: Laptop, Price: 999.99, Stock: 10)

Products sorted by Stock Quantity:
Product(ID: P001, Name: Laptop, Price: 999.99, Stock: 10)
Product(ID: P003, Name: Headphones, Price: 199.99, Stock: 30)
Product(ID: P002, Name: Smartphone, Price: 499.99, Stock: 50)
```

Explanation:

This task demonstrates how searching and sorting are used in real-world retail systems. A Hash Map (dictionary) is used to store products by ID, allowing instant O(1) lookup during billing or stock checking. Sorting operations like arranging items by price or quantity are done using efficient algorithms such as Python's Timsort or Merge Sort. Since shops perform searches more often than sorting, fast lookup is prioritized. This approach improves system speed and scalability for large inventories.

Task-5: Real-Time Stock Data Sorting & Searching

Task Description:

- **Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).**
- **Implement sorting algorithms to rank stocks by percentage change.**
- **Implement a search function that retrieves stock data instantly when a stock symbol is entered.**
- **Optimize sorting with Heap Sort and searching with Hash Maps.**
- **Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.**

Code:

```python
# Task-5: Real-Time Stock Data Sorting & Searching
import heapq

def rank_stocks(stocks):
    heap = []

    for stock in stocks:
        change = ((stock["close"] - stock["open"]) / stock["open"]) * 100
        heapq.heappush(heap, (-change, stock))

    ranked = []
    while heap:
        ranked.append(heapq.heappop(heap)[1])

    return ranked
stocks = [
    {"symbol": "TCS", "open": 3500, "close": 3600},
    {"symbol": "INFY", "open": 1400, "close": 1450},
]

stock_map = {s["symbol"]: s for s in stocks}

def search_stock(symbol):
    return stock_map.get(symbol.upper(), "Stock Not Found")
```

(function) upper: Any

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\App
Assign 12.3.py"
{'symbol': 'TCS', 'open': 3500, 'close': 3600}
```

Explanation:

This task shows how algorithms help analyze live stock market data. Heap Sort (via a priority queue) is useful for ranking stocks based on percentage change, especially when repeatedly selecting top performers. A Hash Map is used to quickly retrieve stock details by symbol, enabling instant search. Both operations are efficient, with sorting taking O(n log n) and searching taking O(1). Such techniques are commonly used in financial dashboards and trading platforms.